

Minimum Swaps to Group All 1's Together II

Target Companies: Josh Tech IBM

Author: Arpit Raj | Institution: LNMIIT Jaipur

Core Concept & Intuition

The objective is to group all 1's present in a binary circular array together with the minimum number of swaps. The problem can be elegantly solved using the **Sliding Window Strategy** by breaking it down into two major steps:

- **Step 1 (Determine Window Size):** The total number of 1's in the entire array determines the exact size of the final contiguous block of 1's. Let this total count be **k**. Therefore, our sliding window size is fixed to **k**.
- **Step 2 (Find Optimal Window):** To minimize the swaps, we need to find a window of size **k** that already contains the **maximum number of 1's**. The number of 0's inside that optimal window represents the minimum swaps needed because each 0 will be replaced by a 1 from outside the window.

MATHEMATICAL FORMULATION

$$\text{Minimum Swaps} = \text{Window Size (Total 1's)} - \text{Maximum 1's in any Window}$$

Example Breakdown: [1, 0, 1, 0, 1]

- **Total 1's:** 3 → Window Size (k) = 3
- **Circular Property:** Since the array is circular, elements wrap around. The possible windows of size 3 are:
 - [1, 0, 1] → contains 2 ones
 - [0, 1, 0] → contains 1 one
 - [1, 0, 1] → contains 2 ones
 - [0, 1, 1] (wrapped) → contains 2 ones
 - [1, 1, 0] (wrapped) → contains 2 ones
- **Maximum 1's found in any window:** 2
- **Result:** Minimum Swaps = 3 - 2 = 1

Handling Circular Arrays (Two Approaches)

Because the array is circular, windows can span across the end of the array back to the beginning. There are two standard strategies to handle this property effectively:

Approach 1: Modulo Arithmetic

Instead of allocating extra memory, we can simulate a circular traversal by iterating the window start from index 0 up to $n + k$. We use the modulo operator ($\% n$) to wrap indices correctly when looking up elements.

- Space Complexity: **O(1)**

Approach 2: Doubled Array Extension

We construct an extended array of size $2 * n$ by appending the original array to itself. This flattens the circular transitions into linear sequences, allowing a

- Time Complexity: **$O(n)$**

standard sliding window to look at all possible circular windows seamlessly.

- Space Complexity: **$O(n)$**
- Time Complexity: **$O(n)$**

Source Code Implementations

Method 1: Sliding Window with Modulo Arithmetic (Space Optimized)

```
class Solution {
public:
    int minSwaps(vector<int>& nums) {
        int n=nums.size();
        //window size utna hi hoga jitna number of ones hoga to lets count;
        int k=0;
        for(int num:nums){
            if(num==1) k++;
        }

        //min swaps = min number of zeroes in window = window size - max number of ones
        int maxu=0;
        int curr=0;
        for(int i=0;i<k;i++){
            if(nums[i]==1) curr++;
        } //first window

        maxu=curr;

        //ab baaki ke windows dekhenge to maximize 1,s

        for(int j=k;j<n+k;j++){ //j<n+k kiya kyunki circular he
            curr+=nums[j%n];
            curr-=nums[(j-k)%n]; //modulo arithmetic to handle circularity
            maxu=max(maxu,curr);
        }

        return k-maxu;
    }
};
```

Method 2: Sliding Window using Doubled Array Extension

```
vector<int> extended(2 * n);

    for(int i = 0; i < 2 * n; i++)
    {
        extended[i] = nums[i % n];
    }

    int windowOnes = 0; for(int i = 0; i < windowSize; i++)
    {
        windowOnes += extended[i];
    }

    int maxOnes = windowOnes;

    // Only consider windows starting in the original array
    for(int l = 1; l < n; l++)
    {
        windowOnes += extended[l + windowSize - 1];
        windowOnes -= extended[l - 1];

        maxOnes = max(maxOnes, windowOnes);
    }

    return windowSize - maxOnes;
};
```